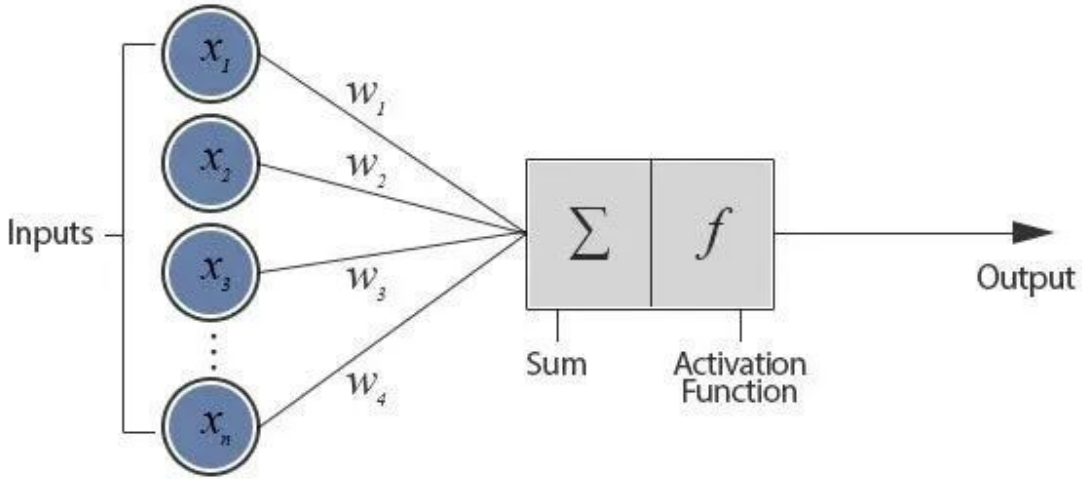
Multi-layer Perceptron

Jayant Kumar Mulmoodi

Perceptron layer ?

The perceptron was also known as a single-layer perceptron as it consisted of only one computational layer – the output layer – while the input layer merely passes features into the system without any processing.



XOR - The Problem

A **single-layer perceptron** (essentially a linear classifier) can only draw a straight line (or hyperplane in higher dimensions) to separate classes. Functions like AND, OR, and NAND are **linearly separable**, so a simple perceptron learns them perfectly.

But **XOR** is **not linearly separable**. Here's why:

The XOR truth table is:

- (0,0) → 0
- (0,1) → 1
- (1,0) → 1
- (1,1) → 0

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

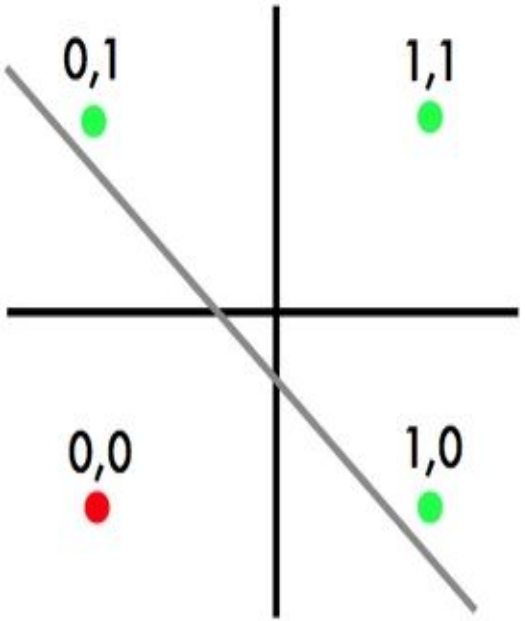
XOR

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

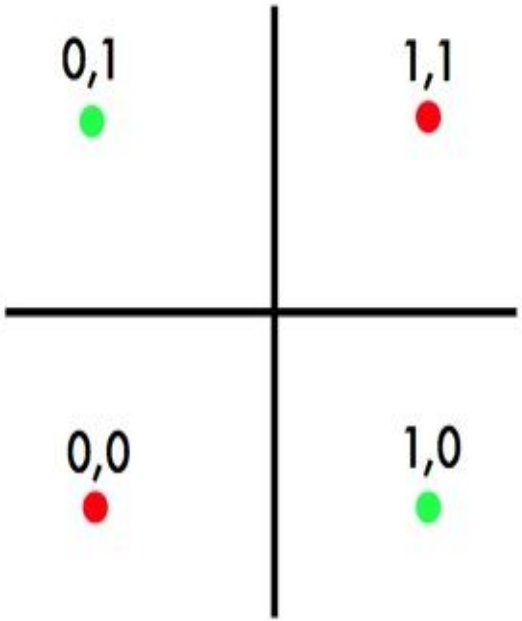
XNOR

XOR killed the single-layer perceptron hype... but gave rise to modern deep learning

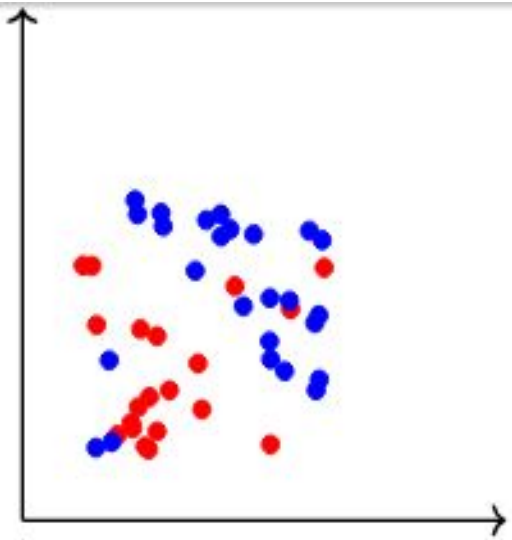
The XOR problem

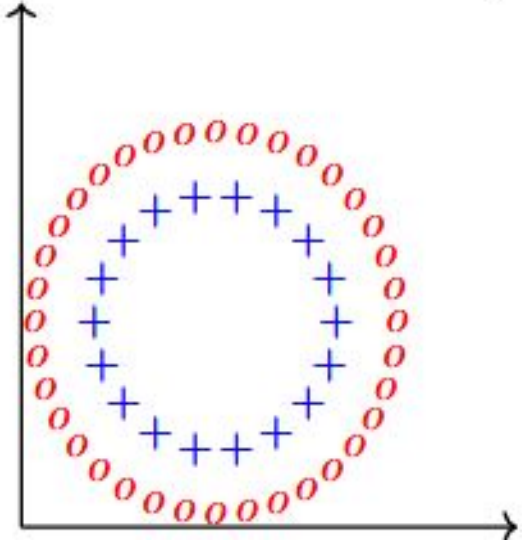


OR



XOR





- Most real world data is not linearly separable and will always contain some outliers
- In fact, sometimes there may not be any outliers but still the data may not be linearly separable
- We need computational units (models) which can deal with such data
- While a single perceptron cannot deal with such data, we will show that a network of perceptrons can indeed deal with such data

Solution for XOR - MLP

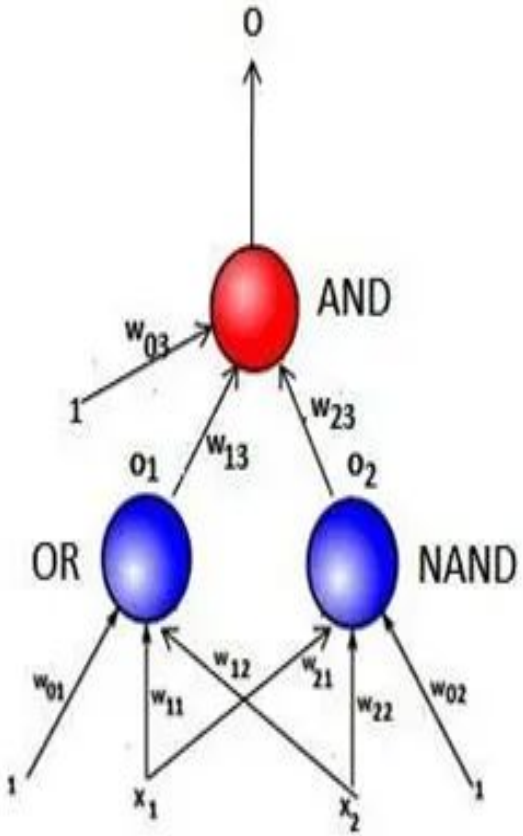
The breakthrough came with **multi-layer perceptrons (MLPs)** — adding one or more hidden layers with **non-linear activation functions** (like sigmoid, tanh, or ReLU).

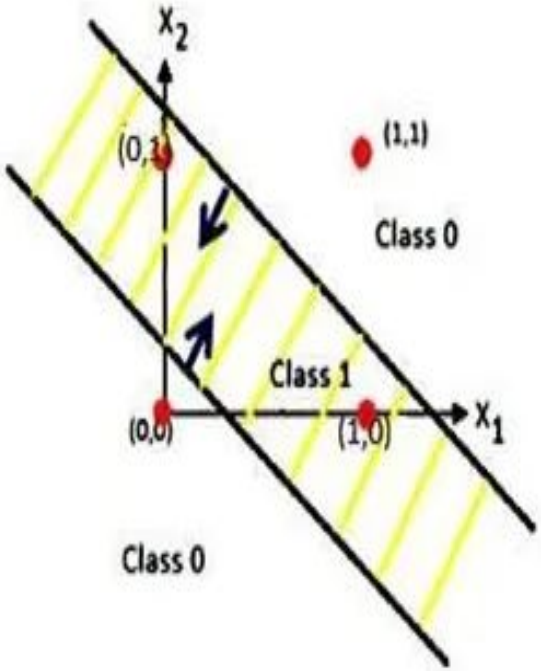
This lets the network learn **non-linear decision boundaries** by combining multiple linear transformations + non-linearities.

A minimal MLP that solves XOR typically has:

- 2 input neurons
- 2 hidden neurons (with non-linear activation)
- 1 output neuron

Solution for XOR - MLP





Truth Table for XOR Gate				
X1	X2	Output, O1 (OR Gate)	Output O2 (NAND Gate)	Final Output O1 (AND) O2
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Pytorch - XOR Implementation

MLP (Multi-Layer Perceptron)

1. A **Multi-Layer Perceptron (MLP)** is the foundational architecture of modern Deep Learning.
2. It is a class of **feedforward artificial neural networks**
3. It consists of at least three layers of nodes: **an input layer**, **a hidden layer,** and **an output layer**

Why Multi-layer Perceptron

Multi-layer perceptrons (MLPs) — adding one or more hidden layers with non-linear activation functions (like sigmoid, tanh, or ReLU).

This lets the network learn **non-linear decision boundaries** by combining multiple linear transformations + non-linearities.

MULTI-LAYER PERCEPTRON: ARCHITECTURE AND TRAINING MODES

ON-LINE TRAINING
(Stochastic)

$[x^{(i)}, y^{(i)}]$

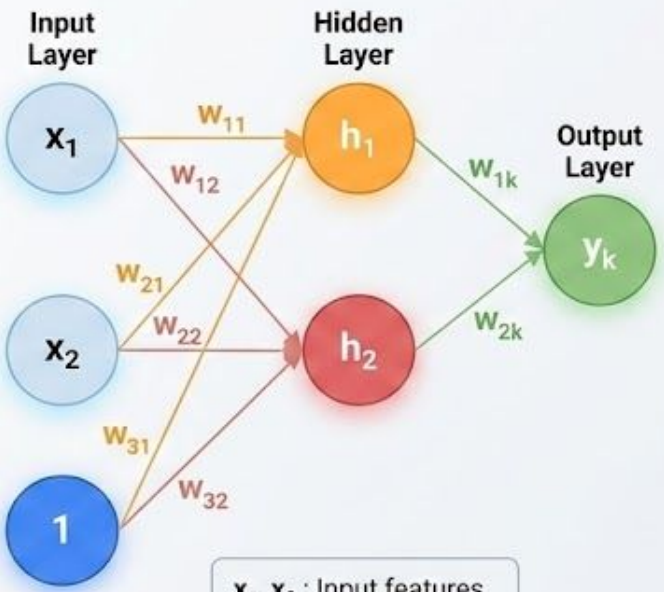
Repeat for every sample in D

Process Single Sample

Compute Prediction (y_{pred})

Calculate Error (**E**)

Immediate Weight Update (**w, b**)



x_1, x_2 : Input features
w_{ij} : Weights
h_1, h_2 : Hidden nodes
y_k : Output variable
1 : Bias node

BATCH TRAINING

$[x^{(1)}, y^{(1)}]$ → $[x^{(2)}, y^{(2)}]$ ⋮ $[x^{(n)}, y^{(n)}]$ →

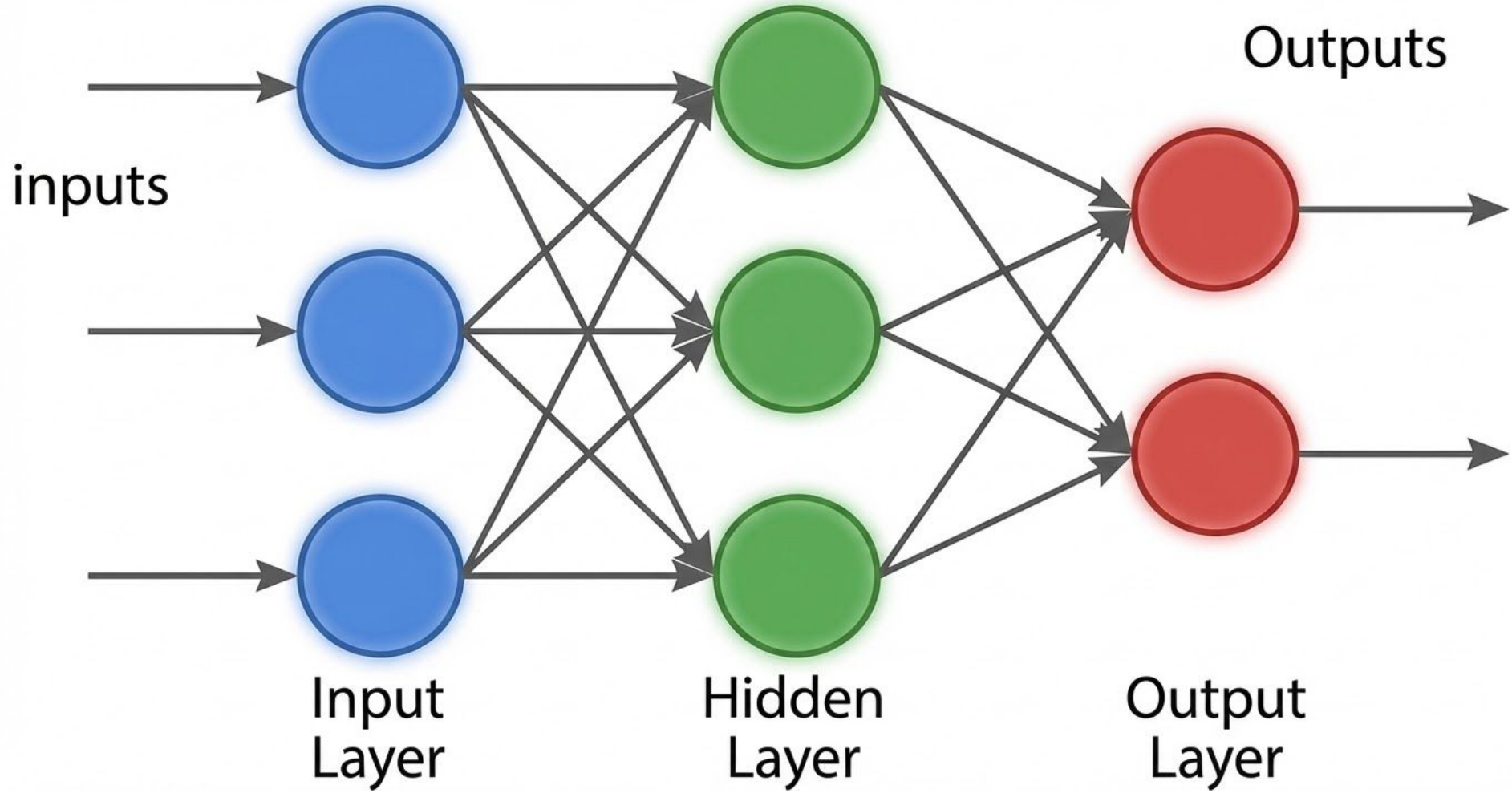
Process Batch of Samples

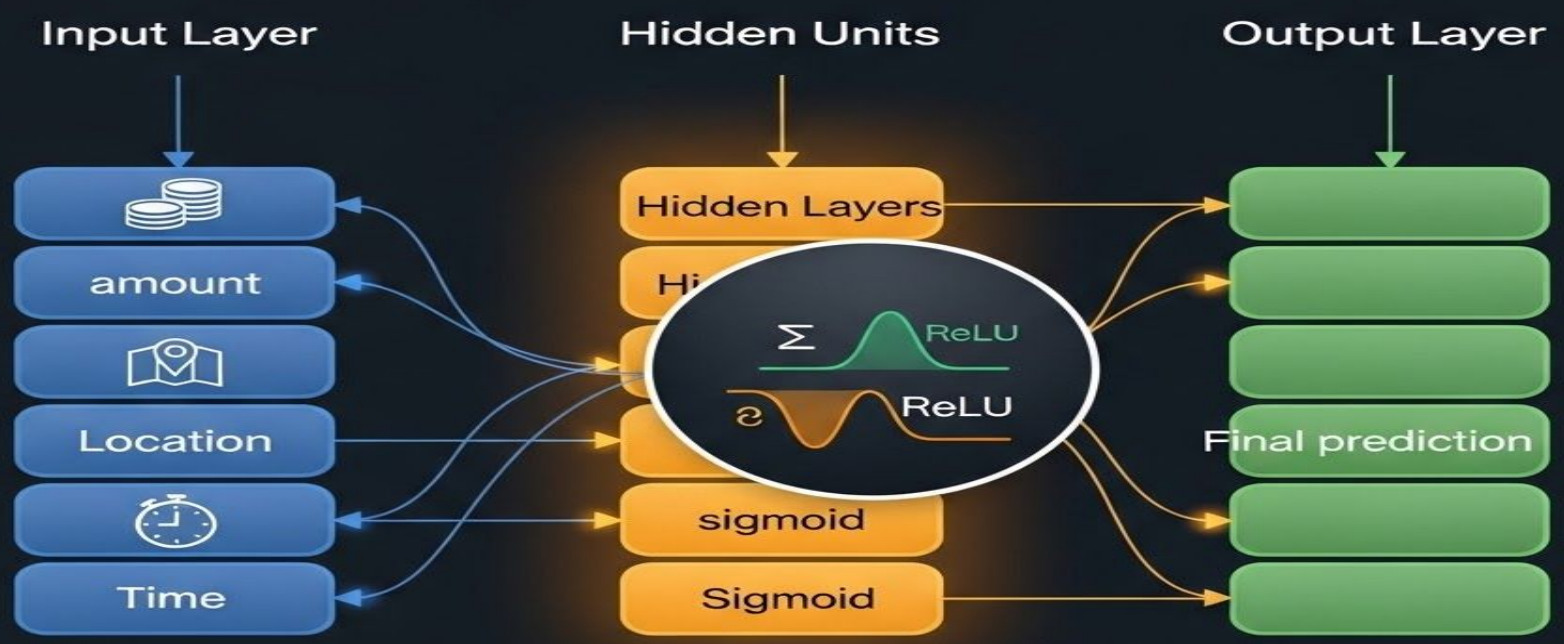
Compute Predictions (y_{pred})

Calculate Errors (**E**)

Accumulate Gradients ($\Delta w, \Delta b$)

Single Weight Update (**w, b**) at end of Epoch


Outputs
inputs
Input
Layer
Hidden
Layer
Output
Layer


Input Layer
Hidden Units
Output Layer
amount
Location
Time
Hidden Layers
ReLU
ReLU
sigmoid
Sigmoid
Final prediction

Layers in multi-layer Perceptron

Input layer

The input layer acts as the entry point for raw data. This layer does not perform any computation. It simply feeds data forward into the network. For example, in credit card fraud detection system, features such as transaction amount, location and time are fed into the model through this layer.

Hidden Units

The perceptrons stacked between the input and output layer are referred to as hidden unit, nodes or neurons. Each hidden unit is a perceptron and contains two key components performing two main operations:

- **Summation**: Computes the weighted sum of inputs using a summation function which forms a linear equation
- **Activation**: Passes the weighted sum through an activation function such as ReLU, sigmoid, tanh, etc. to introduce non-linearity into the system.

Hidden layers (one or more)

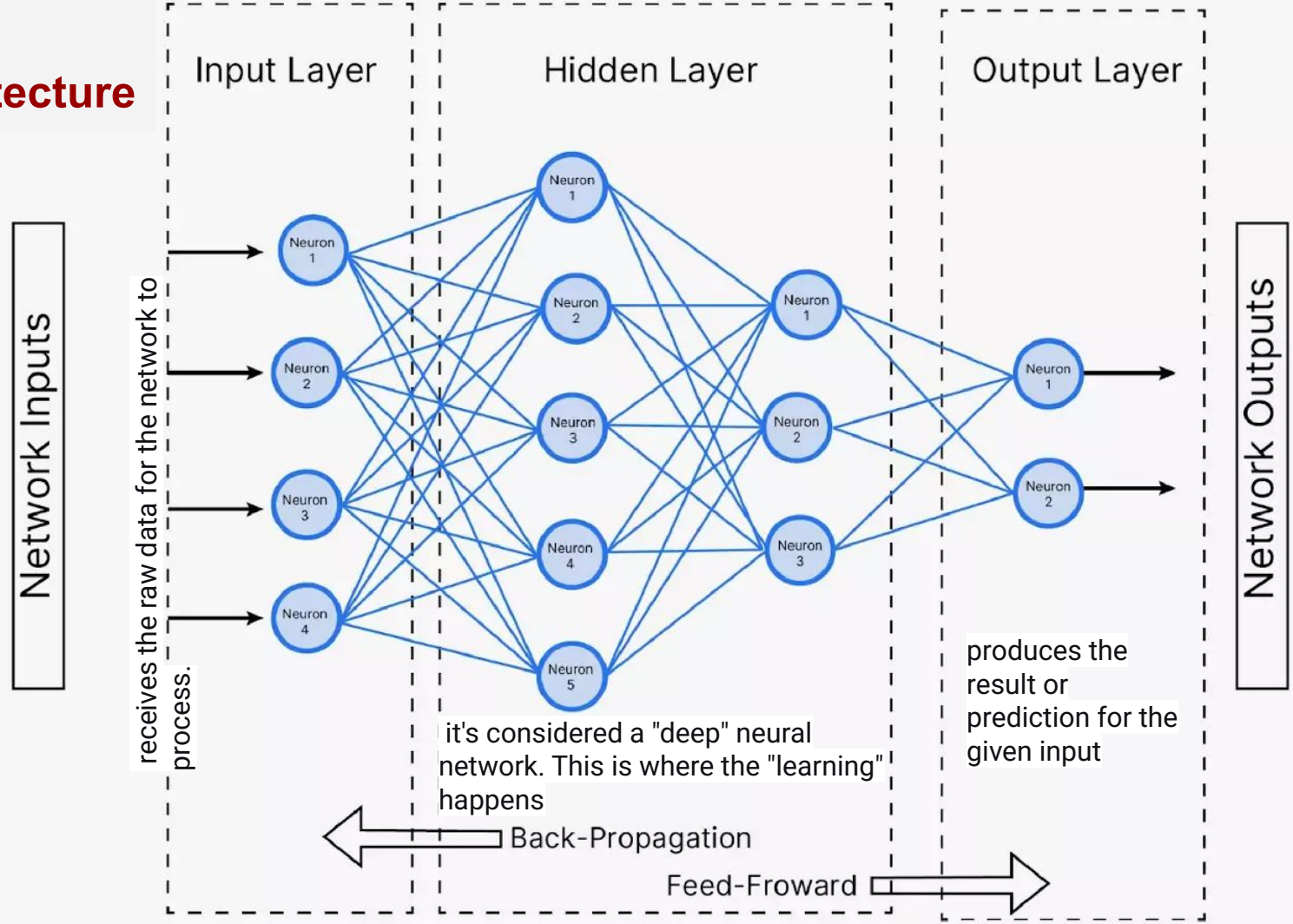
The layers formed after stacking multiple hidden units are called hidden layers. They lie between the input and the output layers. The hidden layers are the real game-changers as they handle all the processing within an MLP. They allow the network to go beyond simple linear data patterns and learn highly complex non-linear patterns in data.

Output layer

The output layer is the final layer of an MLP architecture. It is made up of perceptrons that combine the results of the last hidden layer to produce the final prediction. Each perceptron in this layer also contains two key components:

- **Summation**: Computes the weighted sum of outputs of the nodes of the last hidden layer, forming a non-linear decision boundary.
- **Activation**: Passes the weighted sum through an activation function such as ReLU, sigmoid, or tanh to produce the final output or prediction.

MLP Architecture



How an MLP Works:

An MLP consists of:

- **Input layer** — receives the raw data/features (no computation here, just passes values forward).
- One or more **hidden layers** — where the real learning happens.
- **Output layer** — produces the final prediction (e.g., class probabilities, regression value)

MLP Outputs

For a **regression task** (predicting a continuous value, like a house price), there might be a **single output neuron**.

For a **binary classification task** (e.g., "spam" or "not spam"), there is often a **single output neuron** that outputs a **probability**.

For a **multi-class classification** task (e.g., classifying an image as a "cat," "dog," or "bird"), there would be **output neurons, one for each class**

MLP Hidden Layers

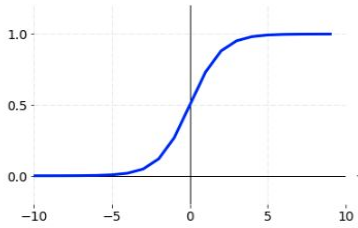
With **non-linear activations**, each layer can create new, more abstract representations:

- Early hidden layers might detect simple patterns (edges, basic combinations).
- Deeper layers combine those into complex concepts (shapes → objects → scenes).

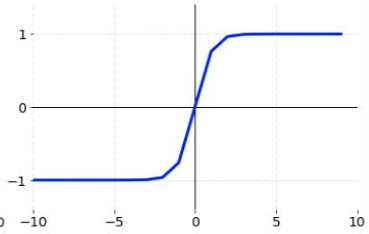
Activation Functions

Basic non-linear functions

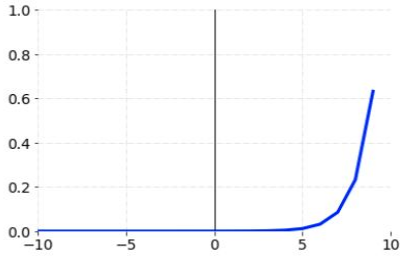
Sigmoid



Tanh



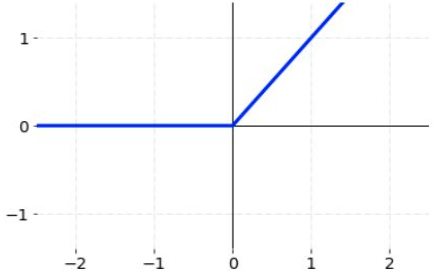
Softmax



Vanishing Gradient

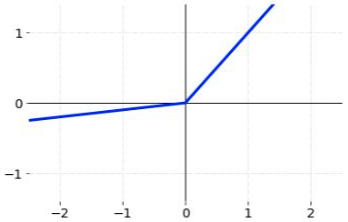
Advanced non-linear functions

ReLU

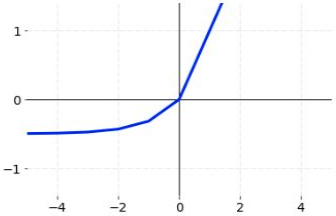


"dying" ReLU

Leaky ReLU



ELU



Bias Term

In a standard Multi-Layer Perceptron (MLP), a bias term is added in every layer except the input layer.

Where Bias Is Present:

Input layer → No bias

Hidden layers → Yes, bias for every neuron

Output layer → Yes, bias for every output neuron

Bias is standard in all computational layers (hidden + output) — every neuron that performs a weighted sum + activation gets its own bias term.

Bias Term - Why in every layer ?

The bias acts as a **learnable offset** or **shift** for the activation function of that neuron:

- It allows the neuron to activate (fire) even if all incoming weighted inputs sum to zero.
- It helps shift the decision boundary / activation curve left or right independently for each neuron.
- Without bias → the model would be forced to fit functions that pass through the origin (much less flexible).
- With bias in every layer → the network gains more expressive power at each transformation step.

Training an MLP : The Learning Process

1. Forward Pass
2. Loss Calculation
3. Backward pass (Backward Propagation)
4. Weight Update (Gradient Descent)

Training an MLP : The Learning Process

Loss Calculation

Regression → **Mean Squared Error** (MSE): $L = (1/n) \Sigma (y - \hat{y})^2$
Binary classification → **Binary Cross-Entropy**
Multi-class → **Categorical Cross-Entropy** (with softmax output)

Backward Pass (Backpropagation)

Backprop uses the chain rule of calculus to compute the gradient of the loss with respect to every weight and bias, layer by layer, backward from output to input.

Training an MLP : The Learning Process

Parameter Update (Gradient Descent)

Use the gradients to update each parameter:

$$\theta \leftarrow \theta - \eta \times \partial L/\partial \theta$$

- η (eta) = **learning rate** (hyperparameter, e.g., 0.001–0.1) — step size
- Negative sign because we move **opposite** to the gradient (gradient points uphill; we want downhill toward lower loss)

Overall Steps

1. Shuffle dataset
2. Split into **batches** (mini-batches, e.g., 32–256 examples)
3. For each batch: forward → loss → backprop → update
4. One pass through entire dataset = **1 epoch**
5. Repeat for many epochs (dozens to thousands) until loss plateaus or validation performance stops improving

Training Improvement Approaches

Batch normalization

Dropout

Learning rate schedules / warm-up

Weight decay / regularization

Forward Pass

Forward Pass (prediction + loss)

- Input → hidden layers → output.
- Compute activations using weights, biases, and non-linear functions (ReLU, sigmoid, etc.).
- Get prediction $\hat{y}$.
- Compute loss $L(\hat{y}, y)$ — e.g., MSE for regression, cross-entropy for classification.

Backward Pass (Backpropogation)

Backpropagation takes this error and propagates it backward layer by layer → figuring out "how much each weight contributed to the final error"

Backward Pass (backpropagation)

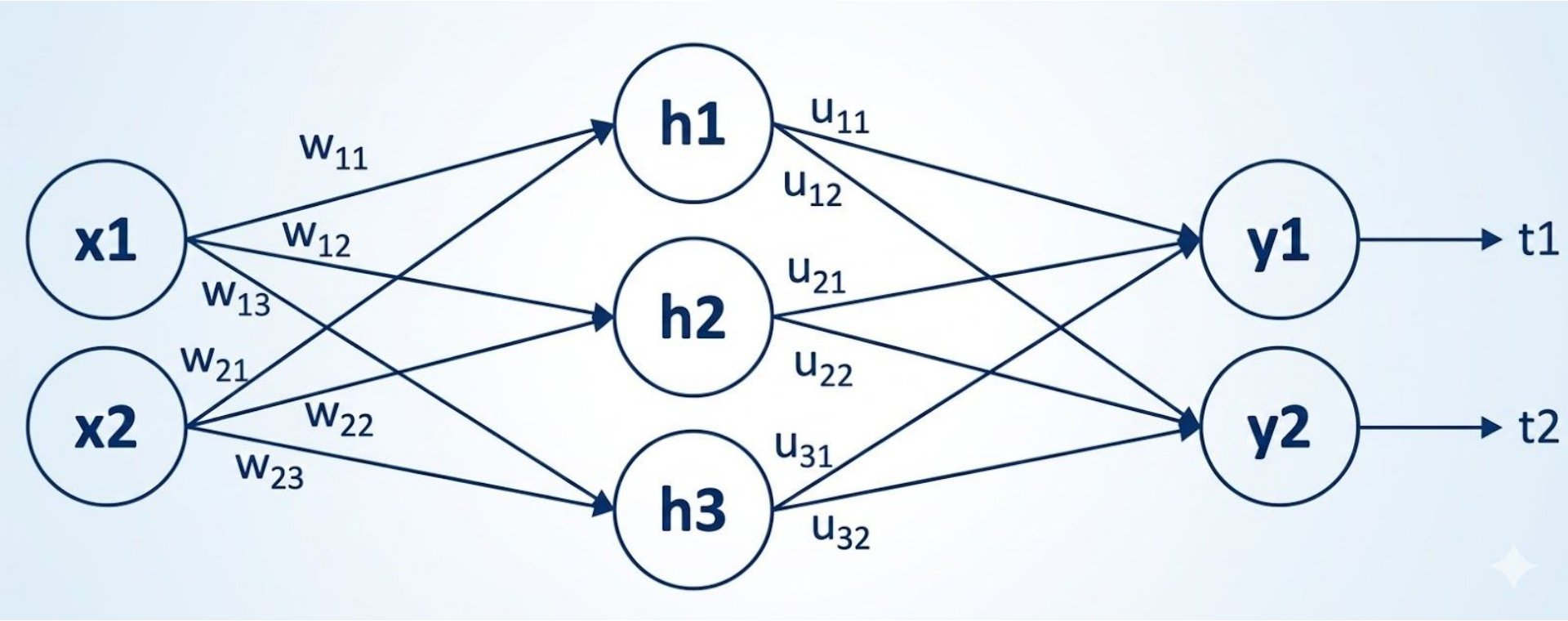
- Start from the output layer: compute how much the loss depends on the output ($\partial L/\partial \hat{y}$).
- Use the **chain rule** to propagate this "blame" (error signal, often called δ) backward to previous layers.
- For each layer, calculate:
  - Gradient w.r.t. activations of previous layer
  - Gradient w.r.t. weights and biases of current layer
- This gives $\partial L/\partial w$ and $\partial L/\partial b$ for **every** parameter.

Parameter Update (Gradient Descent)

Parameter Update (gradient descent step)

- For each weight w and bias b:
  a. $w \leftarrow w - \eta \times (\partial L/\partial w)$
  b. $b \leftarrow b - \eta \times (\partial L/\partial b)$ (η = learning rate)

Backpropagation



The deep neural network above contains an **input layer**, a **hidden layer**, and an **output layer**. It contains two inputs (x_1 and x_2), three hidden units (**nodes**) (h_1,h_2, and h_3), and two outputs (y_1 and y_2).

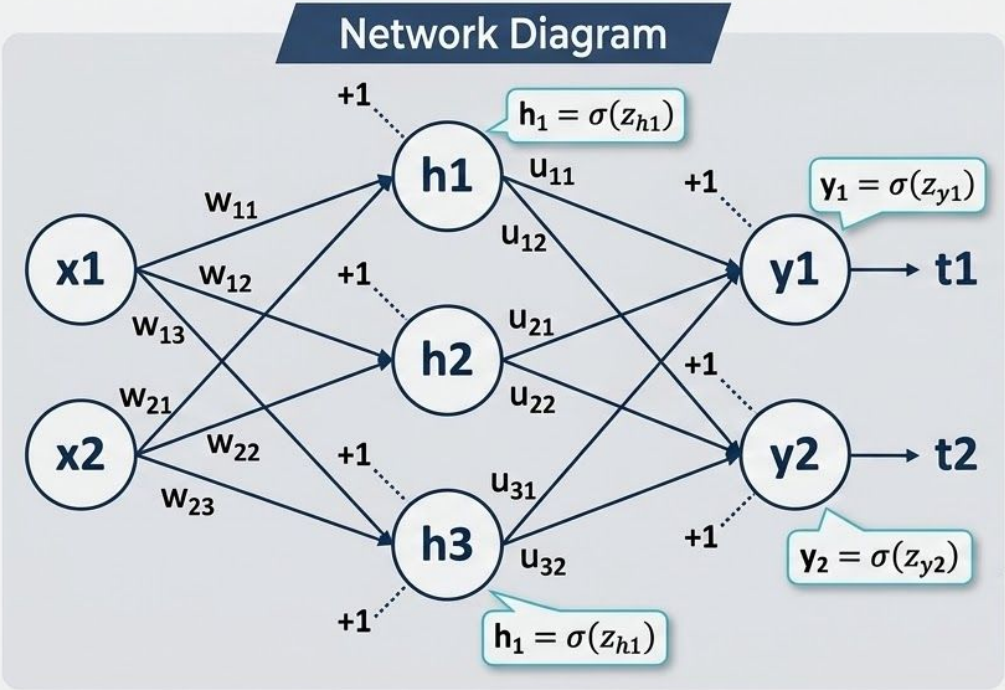
Backpropagation - Network Structure

- **Input Layer to Hidden Layer Weights:**
  - w_{11} connects node x_1 to h_1.
  - w_{12} connects node x_1 to h_2.
  - w_{13} connects node x_1 to h_3.
  - w_{21} connects node x_2 to h_1.
  - w_{22} connects node x_2 to h_2.
  - w_{23} connects node x_2 to h_3.
- **Hidden Layer to Output Layer Weights:**
  - u_{11} connects node h_1 to y_1.
  - u_{12} connects node h_1 to y_2.
  - u_{21} connects node h_2 to y_1.
  - u_{22} connects node h_2 to y_2.
  - u_{31} connects node h_3 to y_1.
  - u_{32} connects node h_3 to y_2.

Output Vectors: After calculation, node y_1 produces output vector t_1, and node y_2 produces output vector t_2.

Backpropagation

Neural Network: Architecture & Sigmoid Activation



The Sigmoid Activation Function ($\sigma(x)$)

The sigmoid (logistic function) is one of the most common non-linearities. It bounds input values between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative Formula: $\sigma'(x)$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

This function is applied at each node of the hidden and output layers.

Computation Example for h1:
$z_{h1} = w_{11}x_1 + w_{21}x_2 + b_1 \Rightarrow h_1 = \sigma(z_{h1})$

Loss functions

Notation and common function definitions:

With this notation, the output y equals the activated linear combination. Since we cannot exhaust all activation and loss functions, we focus on the most common ones: **sigmoid activation** and **L2-norm loss**. Therefor, for the output layer, we have $y = \sigma\left(a^{(2)}\right)$, while for the hidden layer we obtain $h = \sigma(a^{(1)})$.

We obtain this by calculating the sum of the squared differences between the outputs y and the targets t. Its mathematical expression is the following:

$$\textbf{L2-norm loss: } L = \frac{1}{2}\sum_i (y_i - t_i)^2$$

With this notation, the output y equals the activated linear combination. Since we cannot exhaust all activation and loss functions, we focus on the most common ones: **sigmoid activation** and **L2-norm loss**.

Therefore, for the output layer, we have $y = \sigma(a^{(2)})$, while for the hidden layer we obtain $h = \sigma(a^{(1)})$. For the output layer, we calculate the L2-norm loss, which we obtain by calculating the sum of the squared differences between the outputs y and the targets t. Its mathematical expression is the following:

$$\text{L2-norm loss: } L = \frac{1}{2}\sum_i (y_i - t_i)^2$$

A cleaner way to derive the gradient of the sigmoid function, which is often needed, is to directly factor:

$$\begin{aligned}\sigma'(x) &= \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \\ &= \sigma(x) \cdot \left(\frac{e^{-x}}{1+e^{-x}}\right)\end{aligned}$$

Now, notice this identity:

$$\frac{e^{-x}}{1+e^{-x}} = \frac{1+e^{-x}-1}{1+e^{-x}} = 1 - \frac{1}{1+e^{-x}} = 1 - \sigma(x)$$

So, to finalize:

Final Sigmoid Gradient Expression: $\sigma'(x) = \sigma(x)(1-\sigma(x))$

The Linear Model Function and its Application:

Definition

The linear model function equals:

$$f(x) = xw + b$$

where:

x – Input
w – Coefficient (Weight)
b – Intercept (Bias)

Section 2: Multi-Layer Application

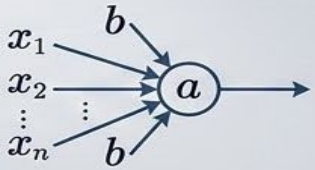
Application

Here, we'll use a for the linear combination before activation:

$$a^{(1)} = xw + b^{(1)}$$

and:

$$a^{(2)} = hu + b^{(2)}$$



h - Input (e.g., from hidden layer)
u - Weights for new layer

Mathematical Notations

Parameter Definitions

- **x**: Input
- **w**: Coefficient (weight)
- **b**: Intercept (bias)
- **h**: Output of the hidden layer, calculated as $h = \sigma(a^{(1)})$
- **u**: Weights for the new layer (used in the calculation of $a^{(2)}$)
- **y**: The final output of the model, calculated as $y = \sigma(a^{(2)})$

- **Layer 1 Linear Combination**: $a^{(1)} = xw + b^{(1)}$
- **Layer 2 Linear Combination**: $a^{(2)} = hu + b^{(2)}$
- **Hidden Layer Output**: $h = \sigma(a^{(1)})$
- **Final Model Output**: $y = \sigma(a^{(2)})$

The Linear Model Function and its Application:

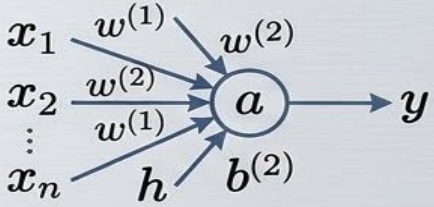
The general linear model function equals:

$$f(x) = xw + b$$

Definitions of general parameters:

x – Input
w – Coefficient (weight)
b – Intercept (bias)

Definition



h - Output from hidden layer
u – Weights for new layer

Section 2: Multi-Layer Application:

Here, we'll use a for the linear combination before activation.
For the input layer, we obtain $a^{(1)} = xw + b^{(1)}$.
For the input layer, we obtain $a^{(1)} = xw + b^{(1)}$.

With this notation, the output y equals the activated linear combination. Since we cannot exhaust all activation and loss functions, we focus on the most common ones: **sigmoid activation** and **L2-norm loss**. Therefore, for the out_e_ layer, we have $y = \sigma(a^{(2)})$,
while for the hidden layer we obtain
$h = \sigma(a^{(1)})$.

Definitions of layer-specific parameters:

- x – Input (general)
- w – Coefficient (weight)
- b – Intercept (bias)
- h – Output from hidden layer
- u – Weights for new layer
- y – Output from outer layer
- $\sigma(a)$ – Sigmoid activation
- $\mathcal{L}$ – L2-norm loss

Backpropagation - Update Parameters Output Layer

We obtain the update rule using the following function:

$$\mathbf{u} \leftarrow \mathbf{u} - \eta \nabla_{\mathbf{u}} L(\mathbf{u})$$

Where η (eta) is the ML algorithm's learning rate.

How do we calculate $\nabla_{\mathbf{u}} L(\mathbf{u})$?

Let's take a single weight u_{ij}. The partial derivative of the loss w.r.t. u_{ij} equals:

$$\frac{\partial L}{\partial u_{ij}} = \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial a_j^{(2)}} \frac{\partial a_j^{(2)}}{\partial u_{ij}}$$

Backpropagation - Update Parameters Output Layer

The first one is the L2-norm loss derivative:

$$\frac{\partial L}{\partial y_j} = (y_j - t_j)$$

The second one is the sigmoid derivative:

$$\frac{\partial y_j}{\partial a_j^{(2)}} = \sigma\left(a_j^{(2)}\right)\ \left(1 - \sigma\left(a_j^{(2)}\right)\right) = y_j\,(1 - y_j)$$

Finally, the third one is the derivative of $a^{(2)} = hu + b^{(2)}$, which equals:

$$\frac{\partial a_j^{(2)}}{\partial u_{ij}} = h_i$$

Backpropagation - Update Parameters Output Layer

Replacing the partial derivatives in the expression above, we get:

$$\frac{\partial L}{\partial u_{ij}} = \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial a_j^{(2)}} \frac{\partial a_j^{(2)}}{\partial u_{ij}} = (y_j - t_j)\, y_j\, (1 - y_j)\, h_i = \delta_j h_i$$

Therefore, we obtain the update rule for a single weight for the output layer using the following:

$$u_{ij} \leftarrow u_{ij} - \eta\, \delta_j\, h_i$$

Backpropagation - Update Parameters Hidden Layer

Similarly to the backpropagation of the output layer, the update rule for a single weight, w_{ij}, is the following:

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \frac{\partial h_j}{\partial a_j^{(1)}} \frac{\partial a_j^{(1)}}{\partial w_{ij}}$$

Backpropagation - Update Parameters Hidden Layer

We use the sigmoid activation and linear model formulas to obtain the following:

$$\frac{\partial h_j}{\partial a_j^{(1)}} = \sigma\left(a_j^{(1)}\right)\ \left(1 - \sigma\left(a_j^{(1)}\right)\right) = h_j\left(1 - h_j\right)$$

and

$$\frac{\partial a_j^{(1)}}{\partial w_{ij}} = x_i$$

Backpropagation - Update Parameters Hidden Layer

Let's take the solution for weight w_{11} as an example:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial y_1} \frac{\partial y_1}{\partial a_1^{(2)}} \frac{\partial a_1^{(2)}}{\partial h_1} + \frac{\partial L}{\partial y_2} \frac{\partial y_2}{\partial a_2^{(2)}} \frac{\partial a_2^{(2)}}{\partial h_1} =$$

$$= (y_1 - t_1) y_1 (1 - y_1) u_{11} + (y_2 - t_2) y_2 (1 - y_2) u_{12}$$

Now, we can calculate $\frac{\partial L}{\partial w_{11}}$, which was the only missing part of the update rule for the hidden layer. The final expression is:

$$\frac{\partial L}{\partial w_{11}} = [(y_1 - t_1)\, y_1\, (1 - y_1)\, u_{11} + (y_2 - t_2)\, y_2\, (1 - y_2)\, u_{12}]\; h_1\, (1 - h_1)\, x_1$$

Backpropagation - Update Parameters Hidden Layer

The generalized form of this equation is:

$$\frac{\partial L}{\partial w_{ij}} = \sum_k (y_k - t_k)\, y_k\, (1 - y_k)\, u_{jk}\, h_j\, (1 - h_j)\, x_i$$

Backpropagation Generalization

We combine the backpropagation for the output and the hidden layers to obtain the general backpropagation formula with the L2-norm loss and sigmoid activations.

$$\frac{\partial L}{\partial w_{ij}} = \delta_j \, x_i$$

where for a hidden layer

$$\delta_j = \sum_k \delta_k \, w_{jk} \, y_j \, (1 - y_j) \, x_i$$

Universal Approximation Theorem

A **feedforward neural network** with at **least one hidden layer** and **a suitable non-linear activation function** (such as sigmoid, tanh, ReLU, etc., as long as it's not a polynomial) can approximate **any continuous function** on a compact (closed and bounded) **subset of $\mathbb{R}^n$ to any desired accuracy** — provided you use enough neurons in the hidden layer(s).

Universal Approximation Theorem

Version / Researcher	Year	Main Point	Activation Requirement	Network Structure
Cybenko	1989	Classic single-hidden-layer result	Sigmoid-like (bounded, continuous, non-constant)	Single hidden layer (wide)
Hornik	1991	Works for almost any continuous non-constant activation	Continuous, non-constant	Single hidden layer
Leshno et al.	1993	Holds **if and only if** activation is **not a polynomial**	Non-polynomial	Single or multi-layer
Modern (ReLU, etc.)	2010s+	ReLU, GELU, Swish, etc. all qualify (non-polynomial)	Non-polynomial	Single hidden layer, but deeper is often more efficient/practical

FeedForward Neural Networks

A Feedforward Neural Network (often abbreviated as FNN or Feedforward NN) is the most fundamental and classic type of artificial neural network. It is also commonly called **a Multi-Layer Perceptron (MLP)** when it has multiple hidden layers, and sometimes referred to as the **"vanilla"** neural network because of its simplicity.

Every neuron in **one layer is fully connected (dense)** to every neuron in the next layer — that's why these are also called **fully connected** or **dense networks**.

Summary

How It Works Mathematically (Forward Pass)

For a network with input **x**, hidden layer activations **h**, and output **ŷ**:

1. Hidden layer (example with one hidden layer): $\mathbf{z_1 = W_1 x + b_1 h = \sigma(z_1)}$ (σ = activation function, e.g., ReLU(z) = max(0, z))
2. Output layer: $\mathbf{z_2 = W_2 h + b_2 \hat{y} = g(z_2)}$ (g = task-specific activation, e.g., softmax for classification)

During training:

- Compute loss L(ŷ, true y)
- Use **backpropagation** to compute gradients ∂L/∂W and ∂L/∂b for all layers
- Update weights with gradient descent (or Adam, etc.)

References

IIT M - Mitesh H Khapra

Internet Resources

AI for Image Generation